CS101: Computer Programming

Introduction to the C Programming Language

Soumitra Ghosh
Department of Computer Science and Engineering

soumitra.cse@iitbhu.ac.in

Learning Objectives

In this lecture you will learn about:

- Algorithm, Pseudocode, Flowchart
- History of C programming language
- Structure of C programming
- Writing your first C Program

What is an Algorithm?

- Steps used to solve a problem
- Problem must be
  - Well defined
  - Fully understood by the programmer
- Steps must be
  - Ordered
  - Unambiguous
  - Complete

Program Development

1. Understand the problem
2. Represent your solution (your algorithm)
    a. Pseudocode
    b. Flowchart
3. Implement the algorithm in a program
4. Test and debug your program

Step 1: Understanding the Problem

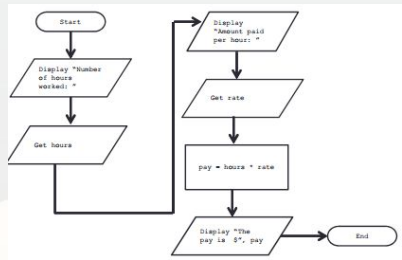
- Input
  - What information or data are you given?
- Process
  - What must you do with the information/data?
- This is your algorithm!
- Output
  - What are your deliverables?

"Weekly Pay" Example

- Create a program to calculate the weekly pay of an hourly employee
  - What is the input, process, and output?
- Input: pay rate and number of hours
- Process: multiply pay rate by number of hours
- Output: weekly pay

Step 2: Represent the Algorithm

- Can be done with flowchart or pseudocode
- Flowchart
  - Symbols convey different types of actions
- Pseudocode
  - A cross between code and plain English
- One may be easier for you – use that one



Step 2A: Pseudocode

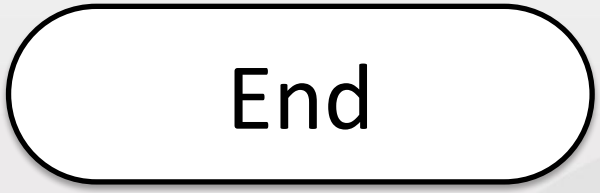
Start with a plain English description, then...

1. Display “Number of hours worked: ”
2. Get the hours
3. Display “Amount paid per hour: ”
4. Get the rate
5. Compute pay = hours * rate
6. Display “The pay is $” , pay

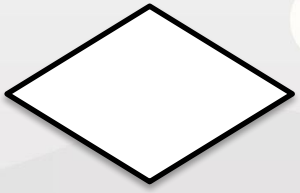
Flowchart Symbols

Start Symbol

Input/Output



End Symbol

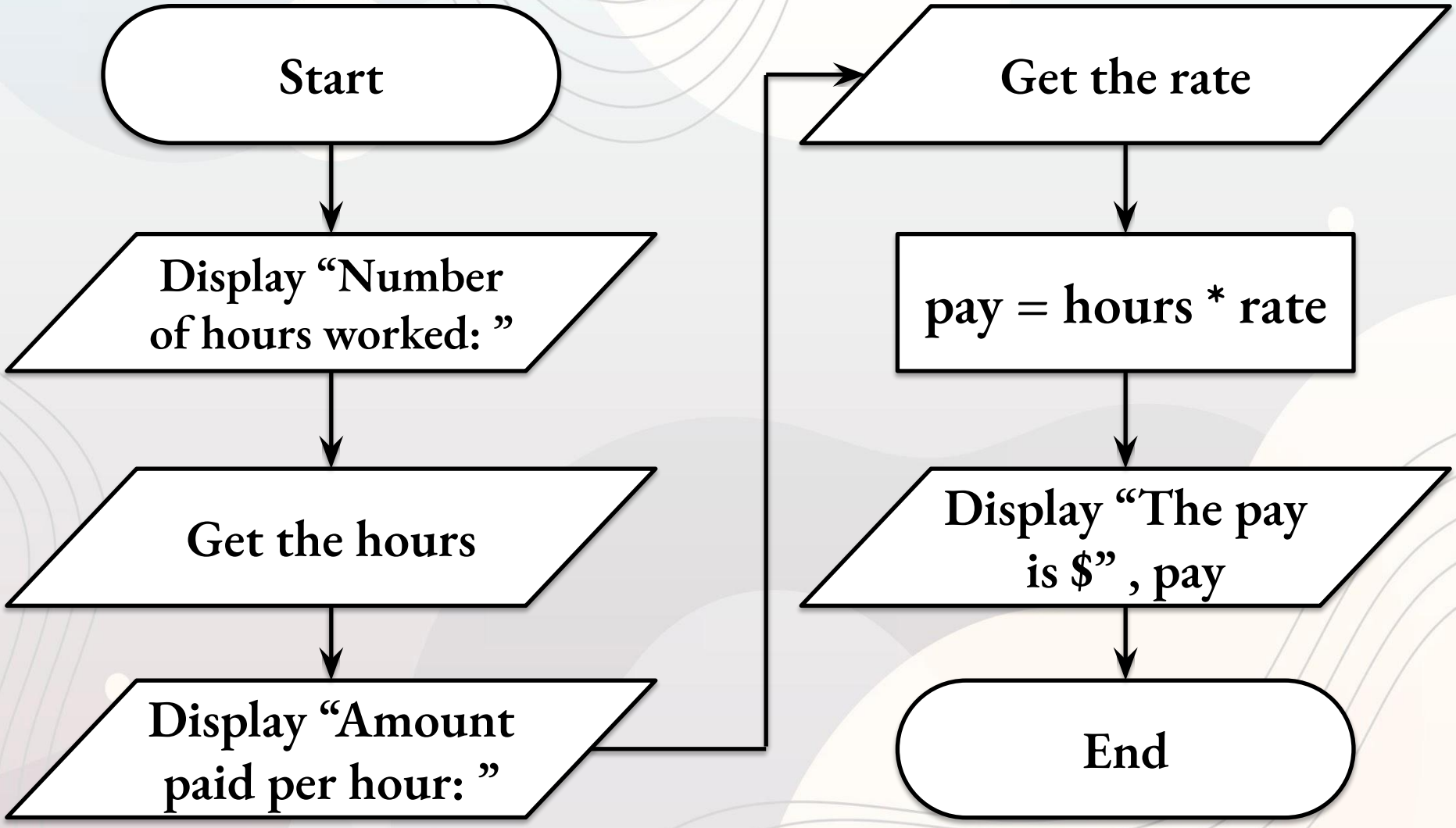


Decision Symbol

Data Processing Symbol

Flow Control Arrows

Step 2B: Flowchart



Is the distinction clear?

Algorithm vs Pseudocode

An **algorithm** is a conceptual, step-by-step logical procedure used to solve a problem.

An algorithm describes **what** needs to be done.

Pseudocode is a text-based, informal way to write down that algorithm using programming-like structures.

Pseudocode acts as a bridge detailing **how** it will look in code.

Algorithms and Language

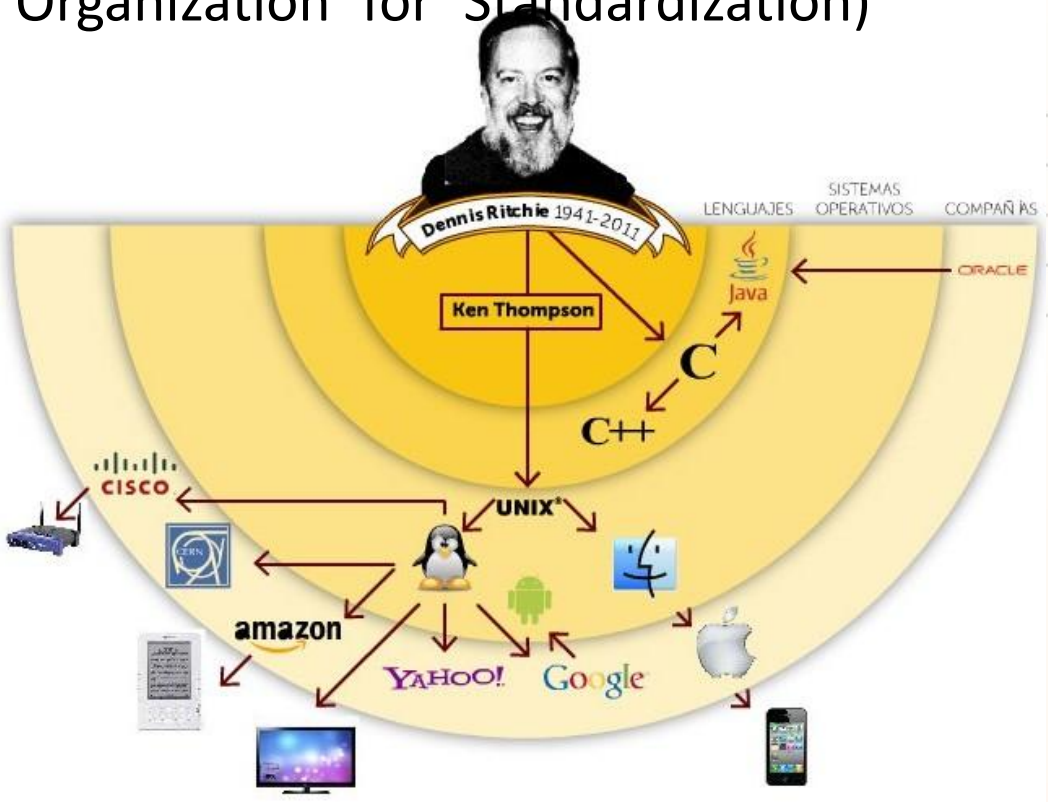
- Notice that developing the algorithm didn't involve any Python at all
  - Only pseudocode or a flowchart was needed
  - An algorithm can be coded in any language
- All languages have 3 important control structures we can use in our algorithms
  - **Sequence** — instructions execute one after another
  - **Selection** — make a decision and choose between alternatives
    e.g., if–else
  - **Iteration** — repeat a set of instructions
    e.g., for, while

Introduction to C Programming Language

- C is a general purpose programming language developed by **Dennis Ritchie** between 1969 and 1972 at Bell Laboratories
  - Originally C was oriented to the implementation of operating systems, specifically **Unix** (with Ken Thompson)
  - First standardized by the ANSI (American National Standards Institute) in 1989
  - Ratified as an ISO (International Organization for Standardization) standard in 1990

Source:
Wikipedia https://en.wikipedia.org/wiki/Dennis_Ritchie

Source: Dennis Ritchie biography by Moisés Cuevas

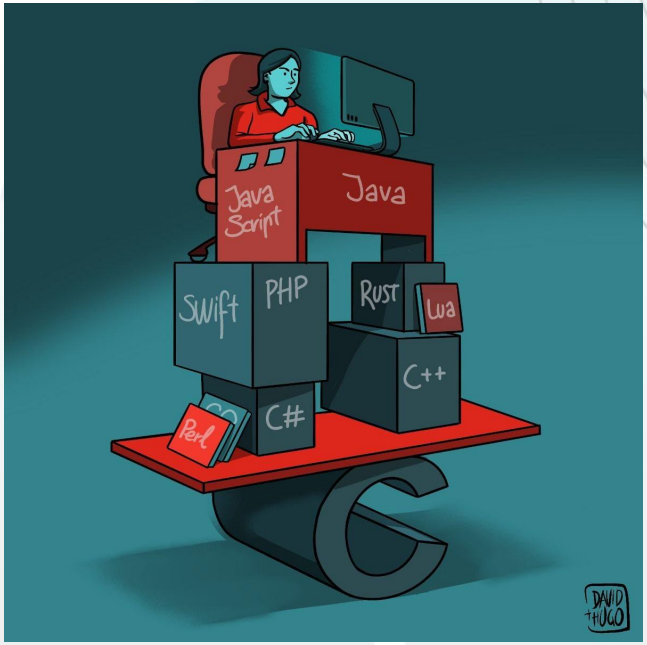


Evolution

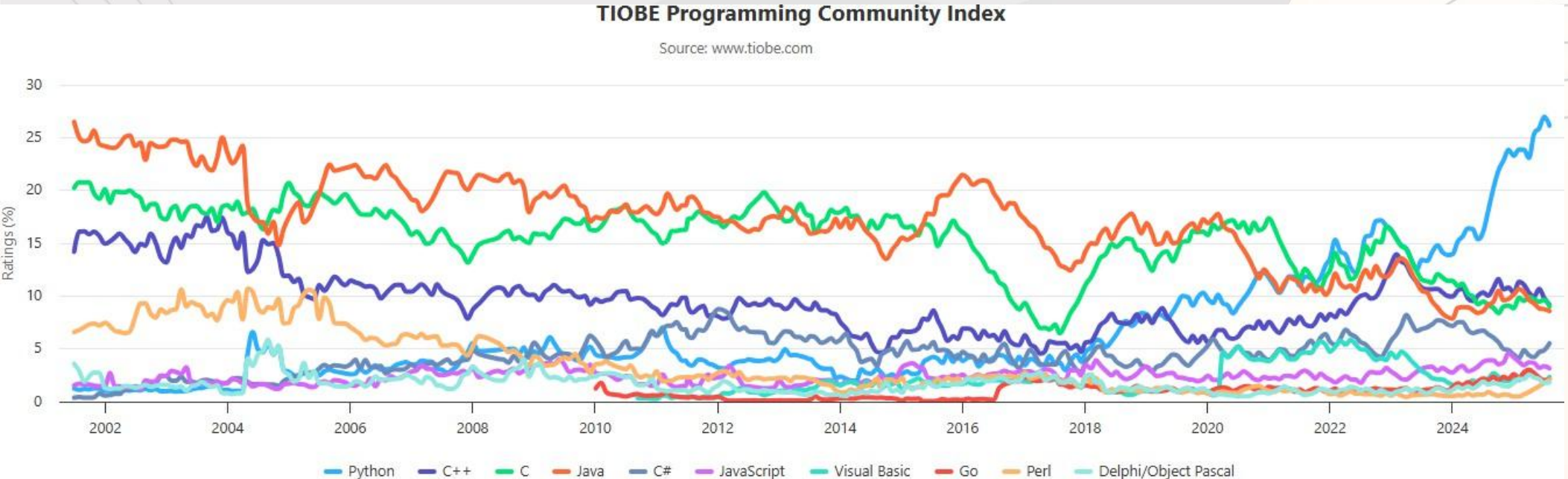
- Thompson designed a small language named B.
- B was based on BCPL, a systems programming language developed in the mid-1960s.
- By 1971, Ritchie began to develop an extended version of B.
- He called his language NB ("New B") at first.
- As the language began to diverge more from B, he changed its name to C.
- The language was stable enough by 1973 that UNIX could be rewritten in C.
- **De facto standard:** Described in Kernighan and Ritchie, *The C Programming Language* (1978)

Popularity of C

- C is considered the foundation of modern programming languages
  - ***C++*** includes all the features of C, but adds classes and other features to support object-oriented programming.
  - ***Java*** is based on C++ and therefore inherits many C features.
  - ***C#*** is a more recent language derived from C++ and Java.
  - ***Perl*** has adopted many of the features of C.



© Image created by Hugo Tobio
Source: Bonilista nº 647, 3 September 2023, La Historia del lenguaje C



Main features of C

- The C programming language can be classified in several ways:
  - C is a **general-purpose** programming language
    - C can be used both **application programming** and **system programming**
  - C is a **high-level** programming language
    - Although it allows certain low level handling (direct memory access)
    - For that reason, it is sometimes classified as a *medium-level* language
  - C is a **compiled** language (the C source code must be converted into machine code to be executed)
  - C is **imperative** (based on statements that are executed sequentially) and **procedural** (it relies on subroutines called *functions* to perform computations)
  - C is **statically-typed** (the type of a variable is known at compile-time) and **weakly-typed** (it allows variables of one type to be used as if they were of another)

Why and Why not?

Properties

- Low-level
- Small
- Permissive

Effectiveness

- Use software tools (lint, debuggers) to make programs more reliable.
- Take advantage of existing code libraries.
- Adopt a sensible set of coding conventions.
- Avoid “tricks” and overly complex code.
- Stick to the standard.

Strengths

- Efficiency
- Portability
- Power
- Flexibility
- Standard library
- Integration with UNIX

Weaknesses

- Programs can be error-prone.
- Programs can be difficult to understand.
- Programs can be difficult to modify.

"Hello IIT BHU" in C

```c
#include <stdio.h>

int main() {
    printf("Hello IIT BHU\n");
    return 0;
}
```

First, we use the compiler `gcc` (GNU Compiler Collection) from the shell to compile a c program (.c file)

```
gcc hello.c
```

Then, we invoke the binary name from the shell to execute it

```
./a.out
```

By convention, the generated executable is called `a.out` by default. We can explicitly set a different binary name using the flag `-o <name>`

```
gcc hello.c -o hello
```

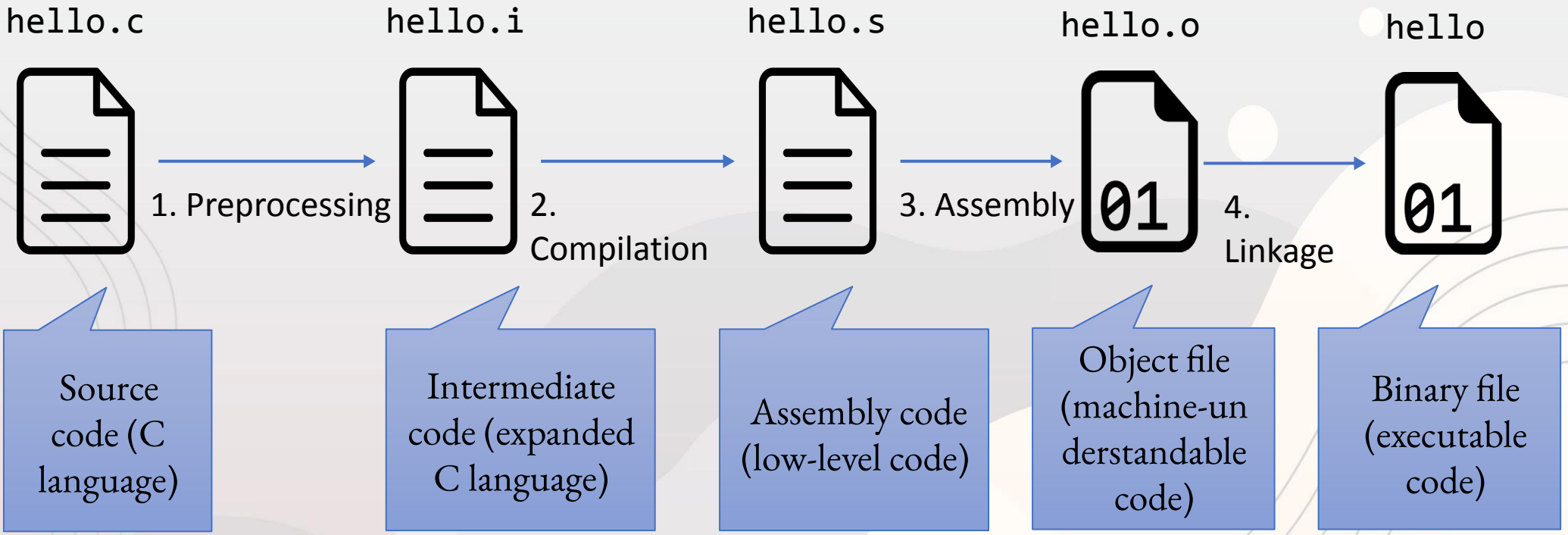
```
./hello
```

Compile-time

Runtime

The build process

- The build process in C has actually 4 stages:

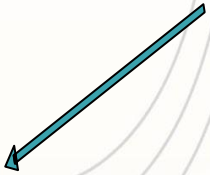


A Simple C Program (simple.c)

```c
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);

    return 0;

}
```

A Simple C Program

C preprocessor directive telling the compiler to include contents of the header file in angle brackets.



```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);

    return 0;

}
```

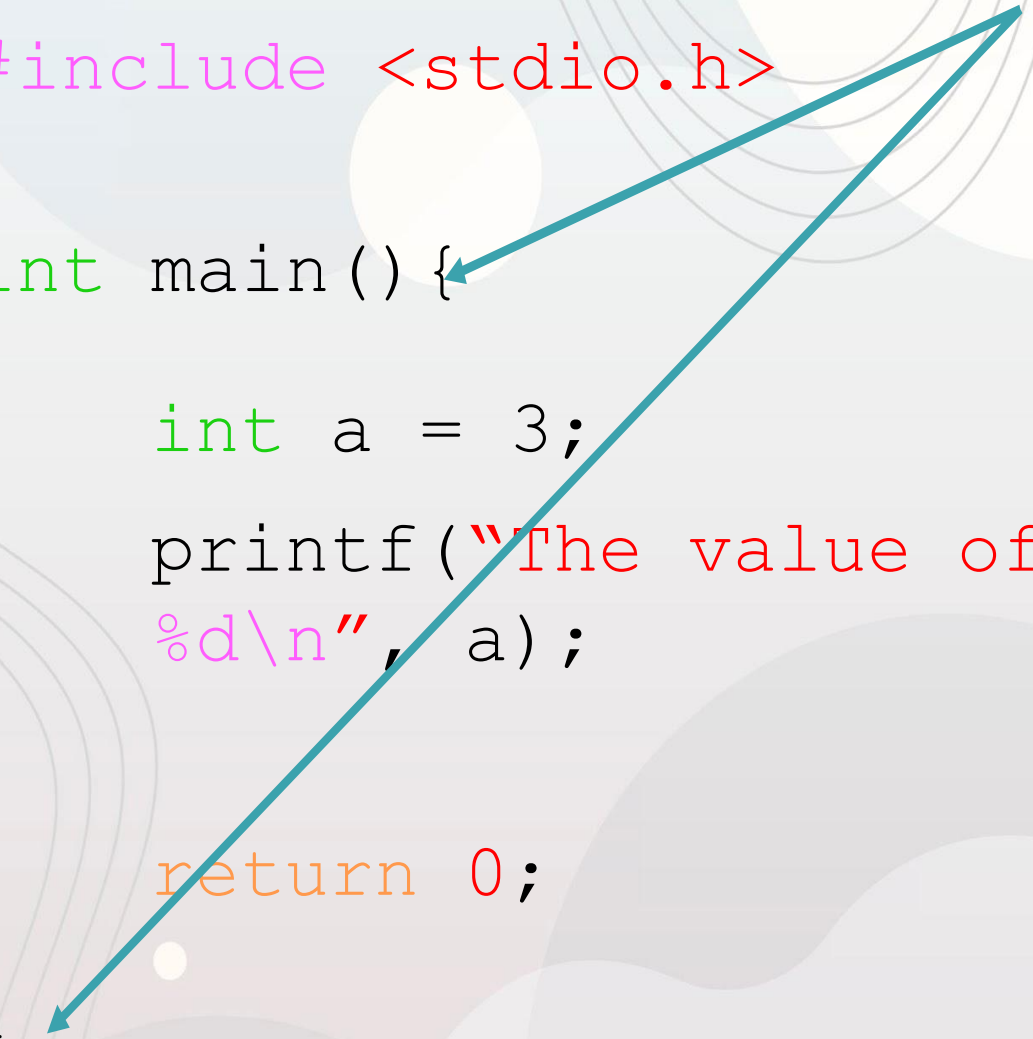
A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);

    return 0;

}
```

Declaration of a function called main, which is where execution of the program begins. The "int" indicates that the function will return an integer value.

A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);


    return 0;

}
```



These curly braces indicate the beginning and end of the main function.

A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);


    return 0;

}
```

Defines an integer called "a" and assigns it a value of 3.

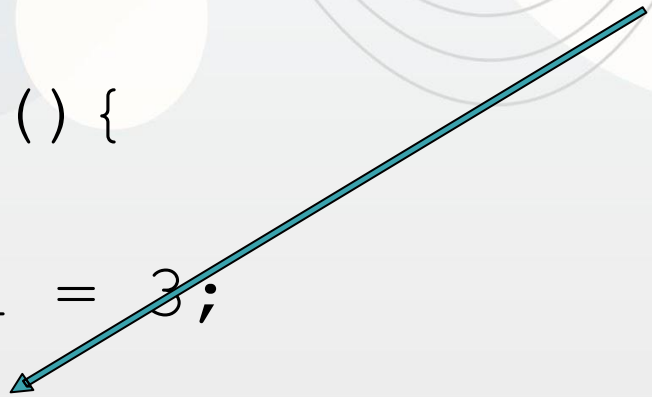
A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);


    return 0;

}
```

A semicolon is used to indicate the end of each statement.

A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);

    return 0;

}
```



A function, called printf, that sends formatted output to stdout (typically the terminal from which the program was run).

This is one of the functions defined in the stdio.h header file.

A Simple C Program

```
#include <stdio.h>

int main(){

    int a = 3;

    printf("The value of this integer is
    %d\n", a);

    return 0;

}
```

Return value "returned" to the run-time environment.

Typically, a value of 0 indicates a normal/successful exit.

A Simple C Program – Ok, let's compile and run

```
$ gcc –o simple simple.c
```

Compile C code into executable

- Using gcc compiler

```
$ ls
simple simple.c
```

-o is a compiler flag that allows you to name the executable

```
$ ./simple
The value of this integer is 3
```

Run program